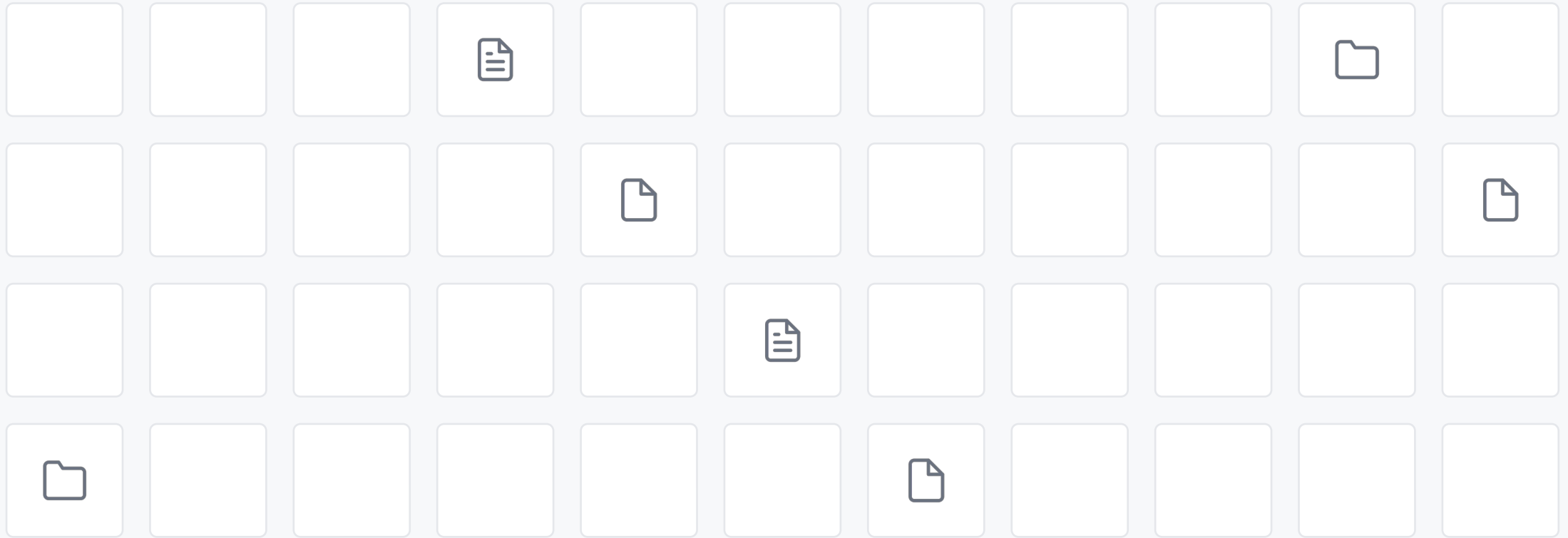


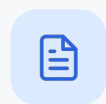


Your knowledge is already there.

Notes, PDFs, wiki pages, and other documents already contain the information we need.

The problem isn't knowledge. It's finding it.

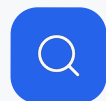
You may have hundreds of documents and thousands of chunks. A question still needs to find the few pieces that actually matter.



- **A QUESTION**



- **MANY DOCUMENT CHUNKS**



- **RELEVANT EVIDENCE**

Example: “how do you uniquely identify a row in a table?” – a real project in Within holds 86 chunks from a single document.



DOCUMENTS → CHUNKS → EMBEDDINGS → RETRIEVAL

Within.

An AI-native knowledge database.

It stores documents, breaks them into searchable chunks, retrieves the relevant evidence, and hands that evidence to an LLM.

The LLM is not the database.

- **KNOWLEDGE**
 - **DATABASE** Stores the actual knowledge.
 - **RETRIEVAL** Finds the pieces relevant to the question.
 - **LLM** Turns those pieces into a useful answer.
 - **ANSWER**
-

The model speaks last.

First: put the knowledge in a database.

PostgreSQL is where Within stores the structured representation of the knowledge — every document, and every chunk of every document.

projects

id
name
description

documents

id
project_id
title
source

document_chunks

content
embedding VECTOR(384)
tsv TSVECTOR

A long document is split into smaller chunks, so retrieval can return the one relevant passage — not the entire document.

DOCUMENT —

CHUNK 1

CHUNK 2

CHUNK 3

CHUNK 4

One chunk. Two ways to search it.

chunk 64 – “A relation is commonly represented as a table. A tuple represents a row in a table...”

FULL-TEXT SEARCH

Looks for words and phrases that literally appear in the text.

Query:

"uniquely identify a row"

Shares the words “row” and “table” with the chunk — a direct lexical match.

Finds it because the words match.

SEMANTIC SEARCH

Looks for meaning, even when the exact words differ.

Query:

"how do you uniquely identify a row?"

Different phrasing, same underlying idea as “a tuple represents a row.”

Finds it because the meaning is related.

Same chunk, two completely different reasons to retrieve it.

What is an embedding?

An embedding represents text as a list of numbers — a point in space that captures what the text means.

"primary key" → [0.12, -0.41, 0.83, ...]

"unique identifier" → [0.15, -0.38, 0.79, ...]

(conceptual example — not real model output; the actual vectors are 384 numbers long)

The exact values don't matter. What matters is the relationship: texts with related meanings end up with nearby representations.

pgvector lets PostgreSQL store these vectors as a native column type, and compare them directly in SQL.

In Within: 384-dimensional, generated locally with all-MiniLM-L6-v2 — no API cost.

One query. Four ways to look.

			
STRUCTURED	FULL-TEXT	SEMANTIC	HYBRID
Uses database fields and filters.	Matches words that appear in the text.	Matches meaning using embeddings.	Combines multiple rankings into one.
<code>structured_retrieve()</code>	<code>fulltext_retrieve()</code>	<code>semantic_retrieve()</code>	<code>hybrid_retrieve()</code>
<code>project, title, date</code>	<code>tsvector, ts_rank_cd</code>	<code>pgvector, cosine distance</code>	Reciprocal Rank Fusion

retrieval.py implements all four as separate, inspectable functions — you can watch them disagree.

WITHIN

Different searches disagree.

"how do you uniquely identify a row in a table"

FULL-TEXT

- 1 **chunk 64** A relation is commonly represented as a table
(one exact-token match)

SEMANTIC

- 1 chunk 61 Here, each row represents a student record
- 2 chunk 65 For example, consider the Student table
- 3 chunk 63 In a relational database
- 4 **chunk 64** A relation is commonly represented as a table
- 5 chunk 67 Student_ID can be the primary key

chunk 64 — #1 in full-text, #4 in semantic. Same evidence, different rank.

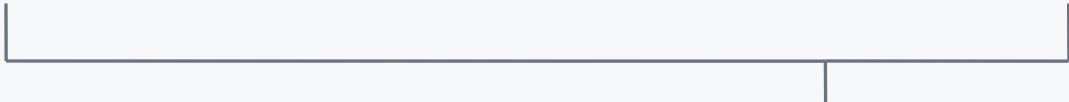
Full-text rewards matching words; semantic rewards related meaning. Neither is wrong.

How do we combine them?

Instead of trying to compare two different score scales, we use each search method's position in its ranking: 1st, 2nd, 3rd...

**FULL-TEXT
RANKING**

**SEMANTIC
RANKING**



COMBINED RANKING

The diagram illustrates the process of combining two rankings. A horizontal line connects the two input rankings. From the center of this line, a vertical line points down to the 'COMBINED RANKING' label. From each end of the horizontal line, a vertical line points up to the 'FULL-TEXT RANKING' and 'SEMANTIC RANKING' labels respectively.

Reciprocal Rank Fusion: documents that rank highly across multiple lists receive a stronger combined position.

One chunk. Two rankings. One fused result.

FULL-TEXT

chunk 64 → #1

SEMANTIC

chunk 61

chunk 65

chunk 63

chunk 64

chunk 67

RRF

$$\text{score}(d) = \sum 1 / (k + \text{rank}_i(d))$$

$k = 60$ — controls how strongly rank position affects the score

chunk 64: $1/(60+1) + 1/(60+4) = 0.0320$ — ranked #1 for appearing in both lists

Now ask it.

- **QUESTION**

- **RETRIEVED EVIDENCE**

The LLM receives the relevant evidence. It does not search the entire database itself.

- **LLM**

- **ANSWER**

`"What is a primary key in a relational database?"`

A primary key is an attribute that uniquely identifies each row in a table — for example, Student_ID, because every student has a unique identifier ①.

An answer should be traceable.

ANSWER

“...every record can be distinguished from all others”

| 1

EVIDENCE

"A relation is commonly represented as a table. A tuple represents a row in a table..." – chunk 64

|

SOURCE

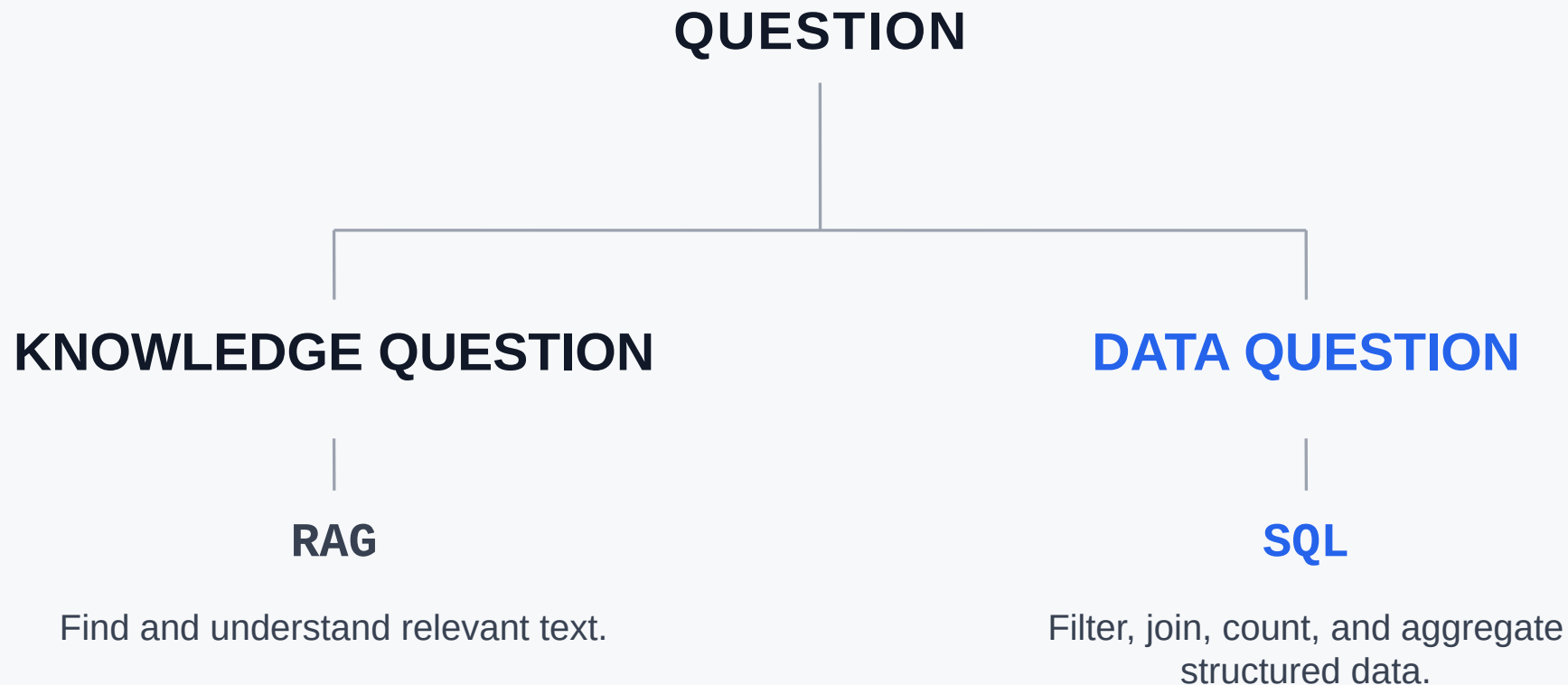
Unit 1 · Database Management Systems

Every cited claim can be traced back to the retrieved source — it's a link, not a footnote.

“Also retrieved”: chunks Within found but didn't cite stay visible too. Nothing is hidden.

Not every question is a retrieval question.

“How many chunks does each document have?” This isn’t a question about interpreting text. It’s a question about structured data.



Ask the database directly.

"How many chunks does each document have?"

SQL

```
SELECT d.id AS document_id,  
       d.title,  
       COUNT(dc.id) AS chunk_count  
FROM documents d  
LEFT JOIN document_chunks dc  
  ON dc.document_id = d.id  
GROUP BY d.id, d.title  
ORDER BY chunk_count DESC  
LIMIT 100
```

RESULT

title	chunks
Unit 1	86
EU Customer Data	60

COUNT() tallies the chunks, GROUP BY groups them per document, ORDER BY ranks the busiest document first.

The model writes the SQL. PostgreSQL performs the calculation.

The database still has to be safe.



SELECT-only

Only read queries are allowed. sqlparse checks the statement type before anything runs.



Single statement

No chained commands. A second statement after a semicolon is rejected outright.



Read-only transaction

SET LOCAL transaction_read_only = on — the database itself refuses to write.



5-second timeout

Long-running or runaway queries are stopped automatically.

Generated SQL is validated before it ever touches the database. Proven directly: DELETE, chained statements, and queries against tables outside the schema are all rejected.

All of it. One system.

PIPELINE

Documents

Chunks

Embeddings

Structured

Full-text

Semantic

Hybrid

RRF · k=60



User



React



FastAPI



PostgreSQL + pgvector

ANSWERS

RAG

Citations

NL → SQL

Validation

Read-only

Result

**Within gives knowledge something
to think with.**